

Análise de Complexidade Assintótica (Big-O)

Exercício 1

Complexidade: $O(1)$

Justificativa: A função apenas verifica se a lista está vazia e acessa o primeiro elemento. Essas operações possuem custo constante, independentemente do tamanho da entrada.

Exercício 2

Complexidade: $O(n)$

Justificativa: O laço percorre todos os elementos da lista uma única vez. O tempo de execução cresce linearmente com a quantidade de elementos.

Exercício 3

Complexidade: $O(\log n)$

Justificativa: A busca binária reduz o espaço de busca pela metade a cada iteração. São necessárias aproximadamente $\log_2(n)$ iterações para encontrar o elemento ou concluir que ele não existe.

Exercício 4

Complexidade: $O(n^2)$

Justificativa: Existem dois laços aninhados percorrendo as combinações de elementos da lista. O número de comparações cresce proporcionalmente a n^2 .

Exercício 5

Complexidade: $O(n^2)$

Justificativa: O primeiro bloco possui custo $O(n)$, enquanto o segundo possui dois laços aninhados, resultando em $O(n^2)$. Pela regra da soma, prevalece o termo dominante $O(n^2)$.

Exercício 6

Complexidade: $O(\log n)$

Justificativa: A variável i é multiplicada por 2 a cada repetição. O número de iterações necessárias para atingir n é proporcional a $\log_2(n)$.

Exercício 7

Complexidade: $O(2^n)$

Justificativa: A versão recursiva do Fibonacci recalcula diversos subproblemas repetidamente. O número de chamadas cresce exponencialmente com n .

Exercício 8

Complexidade: $O(n^2)$

Justificativa: O Bubble Sort utiliza dois laços aninhados. No pior caso, realiza aproximadamente n^2 comparações e trocas.

Exercício 9

Complexidade: $O(n^3)$

Justificativa: A multiplicação de matrizes utiliza três laços aninhados, cada um executando n vezes. Assim, o número total de operações é proporcional a n^3 .

Exercício 10

Complexidade: $O(n \log n)$

Justificativa: O Merge Sort divide o problema em duas metades recursivamente ($\log n$ níveis) e realiza uma intercalação linear em cada nível. O custo total é $O(n \log n)$.